



SR UNIVERSITY
Campus Warangal

Program: II - B. Tech (CS& AI) Batch:

Name:P.Akash

Reg. No:2503A52465

Department: Computer Science and AI Semester: I

AI Assisted Coding-Assignment Date: 6/10/2026

Task 1:

1.Original code:

```
a = 80  
b = 70  
c = 90
```

```
x = (a + b + c) / 3
```

```
if x >= 50:  
    print("pass")  
else:  
    print("fail")
```

prompt:

Refactor this Python code to improve readability. Use meaningful variable names, create a function called `calculate_average()`, improve the formatting, and preserve the original behavior.

AI suggestion :

Replace a, b, and c with meaningful names such as mark1, mark2, and mark3.

Create a reusable `calculate_average()` function.

Use clear formatting.

Keep the pass/fail condition unchanged.

Refactorial code:

```
def calculate_average(first_score, second_score, third_score):
```

```
total = first_score + second_score + third_score  
return total / 3
```

```
score_one = 80  
score_two = 70  
score_three = 90
```

```
average_score = calculate_average(score_one, score_two, score_three)
```

```
if average_score >= 50:  
    print("pass")  
else:  
    print("fail")
```

Test output:

Pass

Explanation:

a, b, c → mark1, mark2, mark3

x → average

The average calculation is placed inside calculate_average().

The formatting is clearer.

The **original and refactored programs produce the same result** , so the original behavior is preserved.

Task2:

Original code:

```
name1 = "Arun"  
marks1 = 85
```

```
if marks1 >= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")
```

```
name2 = "Priya"
marks2 = 72
```

```
if marks2 >= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

Prompt:

Identify the duplicated logic in this Python code and refactor the repeated logic into a reusable function. Test the function with at least four students while keeping the output correct.

AI suggestion :

```
def check_result(name, marks):
```

Refactorial code :

```
def check_student_result(student_name, student_marks):
    if student_marks >= 50:
        print(student_name, "Pass")
    else:
        print(student_name, "Fail")
```

```
students = [
    ("Arun", 85),
```

```
("Priya", 72),  
("Karan", 45),  
("Meera", 60),  
]
```

```
for student_name, student_marks in students:  
    check_student_result(student_name, student_marks)
```

test output:

```
Arun Pass  
Priya Pass  
Rahul Fail  
Meena Pass
```

Explanation:

The original code repeats the same if-else logic for every student.

The refactored code puts this logic inside check_result().

Advantages:

- Less repeated code.

- Easier to maintain.

- If the pass mark changes from 50 to another value, we change it in one place.

- The function can be reused for any number of students.

The output for Arun and Priya remains correct, and the function also works correctly for the additional two students.

Task 3:

Original code:

```
age = 68  
temperature = 39
```

```
if age >= 60 and temperature >= 38:  
    print("High Priority")  
elif age >= 60 and temperature < 38:
```

```
print("Medium Priority")
elif age < 60 and temperature >= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

Prompt :

Simplify this Python conditional logic without changing its behavior. Create a function called `determine_priority()` and make the logic easier to understand. Test it with at least four different age and temperature combinations.

AI suggestions:

```
if age >= 60 and temperature >= 38:
    High Priority
elif age >= 60 or temperature >= 38:
    Medium Priority
else:
    Normal Priority
```

Refactored code:

```
def determine_priority(age, temperature):
    if age >= 60 and temperature >= 38:
        return "High Priority"
    if age >= 60 or temperature >= 38:
        return "Medium Priority"
    return "Normal Priority"
```

```
test_cases = [
    (68, 39, "High Priority"),
    (68, 37, "Medium Priority"),
    (45, 39, "Medium Priority"),
    (45, 37, "Normal Priority"),
]
```

```
for age, temperature, expected in test_cases:
    priority = determine_priority(age, temperature)
    assert priority == expected
    print(f"Age {age}, temperature {temperature}: {priority}")
```

Test output:

```
High Priority
Medium Priority
Medium Priority
Normal Priority
```

Explanation:

The refactored code removes the repeated conditions and combines the two **Medium Priority** cases using or.

The determine_priority() function also makes the logic reusable.

AI preserved the original behavior: Yes. The original and refactored versions give the same result for the tested combinations.

Task 4:

Original code:

```
patient_ids = [101, 205, 309, 412, 518, 623]
target = 412
```

```
found = False
```

```
for patient_id in patient_ids:
    if patient_id == target:
        found = True
```

```
if found:
    print("Patient Found")
else:
    print("Patient Not Found")
```

Prompt :

Improve the readability and efficiency of this Python code. Determine whether a list, set, or dictionary is better for frequent patient ID lookups. Implement the most suitable

option and explain the time complexity.

AI suggestion :

The original code uses a **list** and manually loops through every ID.

For frequent lookups, a **set** is better because:

List lookup: **$O(n)$** average time.

Set lookup: **$O(1)$** average time.

Patient IDs only need to be checked for existence, so a set is suitable.

Refactored code:

```
patient_ids = {101, 205, 309, 412, 518, 623}
```

```
target = 412
```

```
if target in patient_ids:
```

```
    print("Patient Found")
```

```
else:
```

```
    print("Patient Not Found")
```

test output:

Patient Found

Patient Not Found

Explanation:

Data Structure	Typical Lookup
List	$O(n)$
Set	$O(1)$ average
Dictionary	$O(1)$ average

Task 5

Original code:

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]
```

```
total = 0
```

```
for s in students:  
    total = total + s[1]
```

```
avg = total / len(students)
```

```
for s in students:  
    if s[1] >= 50:  
        print(s[0], "Pass")  
    else:  
        print(s[0], "Fail")
```

```
print("Average =", avg)
```

Prompt:

Review this legacy Python code and identify at least five improvements. Refactor it using meaningful variable names, reusable functions, and suitable Python features where they improve clarity. Preserve the original output and behavior.

AI suggestion :

Five improvements suggested:

1. Use meaningful names instead of s, avg, and total.
2. Create a function to calculate the average.
3. Create a function to determine whether a student passes.
4. Use sum() with a generator expression to calculate total marks.
5. Improve formatting and make the code easier to maintain.

Refactored code:

```
students = [  
    ("Arun", 78),  
    ("Priya", 45),  
    ("Rahul", 88),  
    ("Meena", 52)  
]
```

```
def calculate_average(student_list):  
    total_marks = sum(marks for name, marks in student_list)  
    return total_marks / len(student_list)
```

```
def display_result(name, marks):  
    if marks >= 50:  
        print(name, "Pass")  
    else:  
        print(name, "Fail")
```

```
average = calculate_average(students)
```

```
for name, marks in students:  
    display_result(name, marks)
```

```
print("Average =", average)
```

Test output:

```
Arun Pass  
Priya Fail  
Rahul Pass  
Meena Pass  
Average = 65.75
```

Explanation:

The original code works, but it is not very readable or reusable.

s is not a meaningful variable name, so it was changed to name and marks.

avg was changed to average because it clearly explains what the variable contains.

The average calculation was moved into calculate_average(), making it reusable.

The Pass/Fail checking was moved into display_result(), so the logic is separated from the main program.

sum() is used to calculate the total marks more clearly.

The code is formatted into separate sections, making it easier to understand.

The refactored program was tested with the same four students and produced the same results as the original program.

Most useful improvement: Creating reusable functions, because it makes the program easier to maintain and reuse when more students or different calculations are added.